

Supplementary Material: PCU-Select: Amortizing Target-Specific Data Selection Across PEFT Configurations

Anonymous submission

Technical Appendix

Quality Criterion and Break-Even Derivation

The cost comparison applies subject to a user-specified quality tolerance. Let Perf_{PCU} and $\text{Perf}_{\text{LESS}}$ denote the two selectors’ downstream scores. We treat them as tied in quality when

$$\Delta_{\text{quality}} = \text{Perf}_{\text{PCU}} - \text{Perf}_{\text{LESS}}, \quad |\Delta_{\text{quality}}| \leq \epsilon. \quad (\text{S1})$$

Here ϵ is the largest acceptable absolute difference.

Let P and Q denote the numbers of PEFT configurations and tasks, respectively. PCU-Select is cheaper when its shared offline cost plus its PQ online-selection costs falls below LESS’s P datastore-construction costs plus its PQ ranking costs. Solving this inequality gives

$$P > P^* = \frac{C_{\text{offline}}}{C_{\text{LESS}}^{\text{data}} + Q(C_{\text{LESS}}^{\text{rank}} - C_{\text{PCU}}^{\text{pair}})}, \quad (\text{S2})$$

provided that the denominator is positive. For $Q = 4$, PCU-Select’s measured online cost is 1.51 GPU-hours per configuration, or $C_{\text{PCU}}^{\text{pair}} = 1.51/4 = 0.3775$ GPU-hours per PEFT-task pair (displayed as 0.38 in the tables). LESS costs 63.2 GPU-hours per four-task configuration, while PCU-Select pays 144.0 GPU-hours offline plus 1.51 GPU-hours per configuration. Therefore,

$$P^* = \frac{144.0}{63.2 - 1.51} = 2.33. \quad (\text{S3})$$

Because P is integer-valued, PCU-Select costs less at $P \geq 3$. If all four task pairs additionally use 500 reduced calibration labels, the marginal cost becomes $1.51 + 4(1.05) = 5.71$ GPU-hours and $P^* = 144.0/(63.2 - 5.71) = 2.50$; the integer break-even point remains three configurations.

Per-Task Results Behind the Main Table

Table 1 in the main paper summarizes four task-native percentage scores: GSM8K exact match, HumanEval Pass@1, MMLU accuracy, and TyDiQA F1. Because these metrics have different scales, ceilings, and seed variances, their

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

average can hide task-level differences. Table S1 therefore reports them separately. For a baseline b , we define $\Delta b = \text{Perf}_{\text{PCU}} - \text{Perf}_b$, so positive values favor PCU-Select. Each PEFT cell is first averaged over the three target-training seeds; the task-level value then averages the five seen PEFT configurations. For LESS, the table also reports a descriptive 95% interval obtained by resampling the five paired PEFT-level differences. Parentheses give the number of configurations for which PCU-Select has the higher seed-averaged score.

PCU-Select has a higher PEFT-averaged score than RDS+ and Influence on each of the four tasks. Its comparison with LESS varies by task: the paired PCU-Select-minus-LESS interval is positive on MMLU (+0.67 Acc, [+0.24, +1.08]), crosses zero on GSM8K (+0.29 EM, [−0.04, +0.58]) and HumanEval (−0.04 Pass@1, [−0.44, +0.31]), and is negative on TyDiQA (−0.77 F1, [−1.18, −0.43]). These opposing task-level differences yield the registry-level mean gap of +0.04 points reported in the main paper. The observed registry-average performance is therefore close, but the task-level results are heterogeneous.

Task- and configuration-level differences. The largest task-aggregated deficit to LESS occurs on TyDiQA, where PCU-Select is lower by 0.77 F1. The largest configuration-aggregated deficit occurs for MLP-only LoRA (L-r8-mlp), at 0.57 points. Under this configuration, PCU-Select’s HumanEval and TyDiQA results have downstream-seed SDs of 0.68 and 0.62, respectively. These SDs quantify variation across target-training seeds, not uncertainty predicted by the scorer. The results thus localize the largest observed discrepancies to TyDiQA and L-r8-mlp, but do not by themselves identify a causal mechanism.

Evaluation Protocols

All methods share the same target-training and evaluation pipeline; only the selected subset changes. Target training uses the Llama-2-7B backbone (Touvron et al. 2023), freezes the backbone, and trains only the target PEFT. Table S3 lists the per-configuration learning rates (2e-4 for LoRA, 5e-4 for (IA)³, 3e-4 for adapters, and 1e-4 for L-r64-all). The fixed recipe uses AdamW and a cosine schedule with a 0.03 warmup ratio. It runs for 1000 steps with bfloat16, gradient clipping at 1.0, and maximum sequence length 1024. A per-

Task	Metric	PCU	Δ RDS+	Δ Inf.	Δ LESS (95% CI)
GSM8K	EM	22.20	+0.55 (5/5)	+0.58 (5/5)	+0.29 [−0.04, +0.58] (4/5)
HumanEval	Pass@1	17.06	+0.64 (4/5)	+0.24 (4/5)	−0.04 [−0.44, +0.31] (2/5)
MMLU	Acc	49.14	+1.55 (5/5)	+1.00 (5/5)	+0.67 [+0.24, +1.08] (4/5)
TyDiQA	F1	52.30	+0.22 (3/5)	+0.17 (4/5)	−0.77 [−1.18, −0.43] (0/5)

Table S1: Task-level summary of the main 10% budget result. Values are averaged over the five seen PEFT configurations. Δ RDS+ and Δ Inf. are absolute native-point gains over PEFT-agnostic baselines; Δ LESS is the paired difference against per-PEFT LESS with a descriptive 95% cell-resampling interval over PEFT cells. Parentheses report PCU-Select wins among the five PEFT cells for each task; column totals are 17/20, 18/20, and 10/20, respectively.

device batch of 16 over 8 GPUs gives a global batch of 128.

At the 10% budget, the selected subset holds 30K examples. The 1000 steps therefore process 128K samples, or about 4.3 passes over the subset. The dataloader shuffles the full subset each epoch, so it uses every selected example. Selected instructions average ≈ 512 tokens, and length-bucketed batches reduce padding.

With FlashAttention-2 (Dao 2024) and gradient checkpointing, each step takes approximately 5.8 wall-clock seconds. At the observed mean sequence length, this corresponds to about 22 samples/s in aggregate and 1.4K tokens/s per GPU. A 1000-step run takes approximately 96 minutes, or 12.8 GPU-hours under the (wall seconds $\times$ GPU count)/3600 convention. Because all selectors use the same target-training recipe and step budget, we exclude target training from the selection-stage cost comparison.

The primary downstream metric for each task is its standard task-native score, computed by a task evaluator injected into the training runner. We log held-out response-LM loss as an auxiliary diagnostic (`eval_loss`) but use only the primary task metric for comparison. Concrete per-task decoding and scoring protocols are:

- **GSM8K (exact match).** Chain-of-thought generation (Cobbe et al. 2021) with greedy decoding; the evaluator extracts the final numeric answer and checks exact match.
- **HumanEval (Pass@1).** The standard unbiased Pass@1 estimator (Chen et al. 2021): sampling at temperature 0.2 with $n=20$ samples per problem, scored by unit-test execution. This is a multi-sample estimate of Pass@1, not a single greedy sample.
- **MMLU (accuracy).** Log-likelihood scoring (Hendrycks et al. 2021) of the four answer options; the evaluator selects the highest-likelihood option without free-form generation.
- **TyDiQA-GoldP (F1).** Greedy decoding of the answer span (Clark et al. 2020), scored with token-level F1 against the gold answers.

An evaluation adapter supplies these task-specific evaluators to the runner through the `--task-metric-factory` hook. We use the same fixed decoding and scoring settings for every selector; they define the metrics in this study and need not match every public evaluation harness.

Under our evaluation pipeline, untuned Llama-2-7B (Touvron et al. 2023) scores 13.9 on GSM8K, 12.4 on HumanEval, 43.9 on MMLU, and 46.2 on TyDiQA. Our MMLU value is not directly comparable with Meta’s reported 45.3 because the prompt and evaluation harness differ. Every selector-trained model, including Random, exceeds the corresponding untuned score on all four tasks. Random therefore serves as the reference for gains beyond uniform subset selection.

Reproducibility Supplement

This section records the implementation details needed to reproduce the reported experiments. It pins the public releases and states the experiment-specific conversion, sampling, and evaluation rules explicitly.

Data provenance and deduplication. The candidate pool holds 300K mixed instruction examples drawn from seven public instruction corpora. Table S2 identifies the public snapshot, split, record conversion, and final contribution of every source.

We apply one sampling rule to all seven releases. Within each source, a PCG64 generator with seed 0 shuffles source-native IDs (or original row indices when no ID is provided). We scan this order until the source reaches its target quota. FLAN v2, Tulu v2, and OpenOrca preserve the proportions of their eligible rows across FLAN component tasks, Tulu’s `dataset` field, and OpenOrca’s GPT-4/GPT-3.5 partitions, respectively. Largest-remainder rounding converts these proportions to integer stratum quotas. Rejected or duplicate records are replaced by later records from the same source and stratum, so both source and stratum totals remain fixed. We preserve the upstream source ID with every retained record.

Deduplication then runs in three stages. First, it removes exact duplicates by the content hash of the normalized instruction–response pair. Second, it removes normalized-string matches after lowercasing and stripping whitespace and punctuation. Third, MinHash LSH removes near-duplicates using 128 permutations and a Jaccard threshold of 0.8. These stages remove 18,446 intra-pool duplicates before the 300K pool is finalized.

Licensing follows the pinned public releases. The OpenOrca and WizardLM dataset cards record MIT, although upstream terms may also apply. OASST1 uses Apache-2.0, CodeAlpaca uses CC-BY-NC-4.0, and Dolly uses CC-BY-SA-3.0. FLAN components retain their

Source	Public release used	Conversion to an instruction–response record	Final rows
FLAN v2 (Longpre et al. 2023)	Open-Orca/FLAN, third-party FLAN 2022 materialization, rev. 6845b1b3, train partitions	Published inputs $\rightarrow$ instruction and targets $\rightarrow$ response; stratify by component task	110,000
OpenOrca (Lian et al. 2023)	Open-Orca/OpenOrca, default/train, rev. aaeb6859	Concatenate nonempty <code>system_prompt</code> and <code>question</code> ; use response as the target; stratify across the released GPT-4 and GPT-3.5 partitions	70,000
Tulu v2 SFT mix (Iverson et al. 2023)	allenai/tulu-v2-sft-mixture, default/train, rev. 0bc269ee	Retain alternating user/assistant messages and serialize the preceding turns as the instruction; stratify by the released <code>dataset</code> field	50,000
OASST1 (Köpf et al. 2023)	OpenAssistant/oasst1, 2023-04-12 release, default/train, rev. fdf72ae0	Reconstruct English conversation trees and pair each prompt turn with its highest-ranked positively reviewed assistant child	30,000
CodeAlpaca 20K (Chaudhary 2023)	sahil2801/CodeAlpaca-20k, rev. 152bb5e9, all 20,022 source rows	Concatenate instruction and nonempty input; use output as the response	20,000
Dolly 15K (Conover et al. 2023)	databricks/databricks-dolly-15k v1.0, train, rev. bdd27f4d	Concatenate instruction and nonempty context; use response as the target	15,000
WizardLM Evol-Instruct 70K (Xu et al. 2024)	WizardLMTeam/WizardLM_evol_instruct_70k, train, rev. 16b48bd8	Use instruction as the prompt and output as the response	5,000
Total			300,000

Table S2: Candidate-pool provenance, pinned public releases, and record construction. Revision strings are eight-character prefixes of the public repository commits. Counts are final post-filter contributions.

component-specific licenses. The Tulu v2 wrapper is distributed under ODC-BY-1.0, while its constituent sources retain their own terms. We use all non-commercial or mixed-license components only for non-commercial research.

Benchmark decontamination. We decontaminate the pool against the test items of GSM8K, HumanEval, MMLU, and TyDiQA. The filter checks exact matches, normalized-string matches, 13-gram overlap, and MinHash LSH near-duplicates. The 13-gram stage removes candidates with Jacard overlap of at least 0.5; the MinHash stage uses a threshold of 0.8. The filter removes 1,742 pre-finalization candidates (0.58% of the pool size). We then backfill from the same source buckets to keep the final pool at 300K examples, so Table S2 reports final post-filter counts. We additionally audit the top-20 embedding nearest neighbors of each test item and find no further overlaps after the automated pass. All reported results are on the decontaminated pool.

Task sketch construction and HumanEval. Each task sketch contains 32 examples drawn with fixed seed 0 and stratified by response length. For GSM8K, MMLU, and TyDiQA, we sort the eligible training or development records by response character count, split them into three equal index ranges, and use Python’s `random.Random(0)` to draw 11, 11, and 10 examples from the short, medium, and long ranges. Every sketch remains disjoint from its task’s evaluation split.

HumanEval has no official training or development split. Its sketch is therefore source-balanced: 16 examples come from the `full/train` split (374 rows) of MBPP (Austin et al. 2021), public revision 4bb6404f, and 16 come from the CodeAlpaca 20K release pinned in Table S2. For MBPP, `text` is the instruction and the canonical code solution

is the response; the tests are retained for decontamination and auditing but are not appended to either field. Within each source, we filter against the 164 HumanEval evaluation problems, form response-character-length terciles, and use `random.Random(0)` to draw 6, 5, and 5 examples from the short, medium, and long terciles, respectively. Exact content matching and AST-normalized function-signature matching are applied before sampling.

Because the candidate pool also contains CodeAlpaca, we test all 32 sketch items against the pool and exclude every content-hash or AST-signature match before selection; the CodeAlpaca quota is then refilled from its seeded order. Thus the sketch is disjoint from both the HumanEval evaluation set and the candidate pool. We enforce the same disjointness for every selector because the sketch is the shared selection signal.

PEFT registry. Table S3 lists all 17 PEFT configurations: five seen, nine unseen same-family configurations, and three unseen families. The unseen-configuration transfer study evaluates eight held-out targets—L-r32-qkvo, AD-b16, L-r64-all, AD-b256, L-r8-highlayers, PRE-l16, PT-l32, and LN—while the configuration-sensitivity study uses the additional LoRA variants. All use 1000 steps, a cosine schedule, and 0.03 warmup; learning rates vary by configuration. The LN configuration trains the `input_layernorm` and `post_attention_layernorm` scale parameters in every Transformer block and excludes the final model-level RMSNorm.

Intervention sites and gradient signatures. The site space Ω provides a shared coordinate system across PEFT families. It contains 24 sites: 8 layers $\times$ 3 module outputs per layer (`attn_out`, `mlp_out`, and `block_residual`).

Config	Family	Target modules	Capacity	LoRA scaling	LR	Group
L-r8-qv	LoRA	q, v	$r=8$	16	2e-4	seen
L-r16-qkvo	LoRA	q, k, v, o	$r=16$	32	2e-4	seen
L-r8-mlp	LoRA	up, down	$r=8$	16	2e-4	seen
IA3-attnmlp	IA ³	k, v, down	–	–	5e-4	seen
AD-b64	Adapter	block-residual	$b=64$	–	3e-4	seen
L-r4-qv	LoRA	q, v	$r=4$	8	2e-4	unseen-config
L-r32-qkvo	LoRA	q, k, v, o	$r=32$	64	2e-4	unseen-config
L-r64-all	LoRA	q, k, v, o, up, down, gate	$r=64$	128	1e-4	unseen-config
L-r8-lowlayers	LoRA	q, v (low third)	$r=8$	16	2e-4	unseen-config
L-r8-highlayers	LoRA	q, v (high third)	$r=8$	16	2e-4	unseen-config
L-r16-hlr	LoRA	q, k, v, o	$r=16$	32	5e-4	unseen-config
AD-b16	Adapter	block-residual	$b=16$	–	3e-4	unseen-config
AD-b256	Adapter	block-residual	$b=256$	–	3e-4	unseen-config
IA3-attnonly	IA ³	k, v	–	–	5e-4	unseen-config
PRE-116	Prefix	attention	len=16	–	2e-4	ood-family
PT-132	P-Tuning	attention	len=32	–	2e-4	ood-family
LN	LN Tuning	input/post-attn RMSNorm	2 scales/layer	–	2e-4	ood-family

Table S3: Full PEFT registry (all 17 configurations). “LoRA scaling” is the LoRA α hyperparameter and is distinct from the intervention-site weight $\tilde{w}_p^\omega$ defined in the main paper; it applies only to LoRA rows. Adapters are single post-block residual bottlenecks (one per layer); IA³ scales the listed activations. LN Tuning updates per-block RMSNorm scales (Wang et al. 2022; Touvron et al. 2023). Learning rates for the three OOD-family targets use the registry default (2e-4). Layer placement is all layers unless noted; low/high-third placements use the bottom/top $\lceil n/3 \rceil$ layers of the 32-layer backbone.

Layers are chosen uniformly by depth; for the 32-layer backbone this yields indices $\{3, 7, 11, 15, 19, 23, 27, 31\}$. Each site carries a Johnson–Lindenstrauss random projection (Johnson and Lindenstrauss 1984) $\Phi_\omega \in \mathbb{R}^{256 \times 4096}$ with entries drawn $\mathcal{N}(0, 1/256)$ under a per-site seed `SHA256(site, global_seed=0)`; projected gradients are L_2 -normalized per row. Each sample’s signature contains 24×256 values. In float16, these projected gradients occupy approximately 3.69 GB (3.43 GiB) for the 300K-example pool. They are stored as a memory-mapped array and reused during offline proxy-label construction; online PCU-Select scoring uses cached z_x features rather than this gradient store. A configuration activates sites through its mask and the capacity factor $\kappa(p, \omega) = \tanh(\eta \log(1 + \text{cap}(p, \omega)/d_{\text{model}}))$, with $\eta=1.0$. Operator type enters the explicit PEFT code rather than a scalar site prior; Table S8 gives the family-specific capacity mapping.

Conditional scorer. The conditional scorer estimates a sample’s utility for a specific task and PEFT configuration. It has three input towers: sample $z_x \in \mathbb{R}^{848}$, PEFT condition $z_p \in \mathbb{R}^{128}$, and task sketch $z_t \in \mathbb{R}^{848}$. Each tower uses a LayerNorm–Linear–GELU–Linear–LayerNorm MLP. Let $h_x, h_t \in \mathbb{R}^{256}$ and $h_p \in \mathbb{R}^{128}$ denote the three tower outputs. The concatenated condition $[h_p; h_t]$ produces 256-dimensional FiLM scale and shift vectors (Perez et al. 2018), which modulate h_x . In parallel, a bilinear layer maps (h_x, h_p) to a 64-dimensional interaction vector. The concatenated 320-dimensional representation passes through a Linear–GELU head that maps it to 256 dimensions. Two linear output heads then predict mean utility $\hat{\mu}_\phi$ and an example-dependent uncertainty scale $\hat{\sigma}_\phi$; the latter uses softplus with a floor of 10^{-3} .

We use $\hat{\sigma}_\phi$ as a ranking penalty. On held-out labels, its predicted standard deviation tracks realized error with a 10-bin expected calibration error of 0.043. In the downstream ablation, setting the uncertainty penalty to zero lowers the reported average by 0.28 points (Table S9).

Training separates cheap proxy learning from expensive short-update supervision. Phase A pretrains on the gradient proxy for 3 epochs at $\text{lr } 3 \times 10^{-4}$ with loss weights (rank, reg, proxy, unc) = (0, 0, 1, 0). Phase B jointly fine-tunes for 2 epochs at $\text{lr } 10^{-4}$ with weights (1.0, 0.3, 0.5, 0.2). Both phases use AdamW, batch size 256, and gradient clipping at 1.0. The four losses are a BPR-style pairwise ranking loss, a Huber regression loss against short-update labels, a Huber proxy-distillation loss against the gradient-proxy labels, and a Gaussian negative-log-likelihood uncertainty loss. We track ranking NDCG on a held-out split of PEFT/task pairs that excludes two configurations and one task from Phase B. We keep the Phase-B checkpoint with the best held-out NDCG. The scorer uses no dropout.

Selection, clustering, and quotas. At application time, the scorer ranks each example by $q = \mu - \lambda \sigma$ with uncertainty penalty $\lambda=0.2$. The penalty favors examples with high predicted utility and lower uncertainty. Ranking alone can concentrate the subset in a few semantic regions, so PCU-Select adds cluster-level coverage. It applies Mini-Batch k -means (Sculley 2010) to the joint embeddings. It uses $k = \max(50, \lfloor \sqrt{N} \rfloor)$, batch size 4096, $n_{\text{init}}=3$, and random state 0. Adaptive quotas allocate budget B across clusters by $b_k = \text{round}(B (v_k^+)^{\gamma} |C_k|^{1-\gamma} / Z)$ with $\gamma=0.6$. Here v_k is the mean conservative score q among the top 10% of examples in cluster C_k , $v_k^+ = \max(v_k, 0)$, and $Z = \sum_j (v_j^+)^{\gamma} |C_j|^{1-\gamma}$. The algorithm distributes remain-

Level	Definition	Evaluation mode
L0	near-support	zero-shot selection
L1	far-support	200–500 labels (0.42–1.05 GPU-h)
L2-LN	unseen family; labels available	200–500-label calibration
L2-Prompt	Prefix/P-Tuning	200–500-label calibration

Table S4: Transfer-evaluation protocol for each registry-defined support level. Costs are per PEFT–task pair, in addition to 0.38 GPU-hours for online selection.

ders by descending weight and trims over-allocation from the lowest-weight clusters.

Targets far from the seen registry can additionally calibrate the scorer with a small label set. The calibrated mean is $\hat{\mu}_{\text{cal}} = \hat{\mu}_{\phi} + W_{\text{cal}}[z_x; z_p; z_t] + b_{\text{cal}}$. We freeze the shared scorer and fit only the zero-initialized residual parameters W_{cal} and b_{cal} for 200 epochs at $\text{lr } 5 \times 10^{-3}$ (Adam, MSE). These deployment calibration labels $u_{\text{cal}}^{\text{hi}}$ are a reduced form of the offline short-update labels u^{hi} . They use only the warm anchor and horizon $h=1$, rather than both anchors and both horizons. Each label therefore costs one quarter of the full offline routine in Eq. (S4). An offline label costs $84.0/10,000 = 0.0084$ GPU-hours, while a calibration label costs ≈ 0.0021 GPU-hours. Budgets of 200 and 500 labels therefore cost 0.42 and 1.05 GPU-hours per PEFT–task pair. The full routine would cost about four times more. We use 200 or 500 $u_{\text{cal}}^{\text{hi}}$ labels for the OOD study.

Structural support levels. The structural support levels determine whether a PEFT code that differs from the training registry is evaluated zero-shot or with calibration. The reported $z_p \in \mathbb{R}^{128}$ contains only the site/operator mask, capacity, and recipe vectors. Prefix, P-Tuning, and LN Tuning use the same deterministic encoding rules as the seen families; no centroid, learned family lookup, or functional fingerprint enters the reported model. We fixed this registry-relative taxonomy before evaluating the held-out targets. **L0** contains near-support same-family targets (L-r32-qkvo and AD-b16). **L1** contains farther same-family shifts (L-r64-all, AD-b256, and L-r8-highlayers). **L2** contains unseen families (PRE-116, PT-132, and LN). These levels define registry-relative structural support. L1 and L2 targets use the reduced calibration protocol described above; their results test whether a small target-specific label set can repair a larger structural shift.

Gradient-proxy and short-update labels. Short-update supervision concentrates expensive labels where they provide different kinds of information. It samples 10,000 triples using a 0.5/0.3/0.2 split. Coverage sampling spans clusters, PEFT families, capacity buckets, and tasks. Uncertainty sampling prioritizes large $\hat{\sigma}_{\phi}(1 + 0.5 \text{ReLU}(\hat{u}_{\text{lo}}))$, while boundary sampling targets disagreement between predicted and observed ranks. Each label measures a short-update loss reduction on the sketch from two frozen anchors— θ_{base} (the pre-trained backbone) and θ_{warm} (a rank-8 attention-only LoRA trained for 200 steps at $\text{lr } 10^{-4}$). Each short update Adapt_p^h

uses one sampled example and 7 in-bucket fillers. The fillers stabilize the update relative to a single-example step. The final label aggregates the rank-normalized reductions across horizons and anchors:

$$u^{\text{hi}} = \frac{1}{|A|} \sum_{a \in A} \sum_{h \in \{1,4\}} w_h \tilde{\Delta}_{a,h}. \quad (\text{S4})$$

It uses weights $w_1=0.4$, $w_4=0.6$ and anchors $A = \{\text{base, warm}\}$. The term $\tilde{\Delta}_{a,h}$ is the reduction defined by the main paper’s short-update equation, rank-normalized within its PEFT, task, anchor, and horizon bucket.

Cost model and seeds. We measure all costs on $8 \times \text{NVIDIA H20-96GB}$ (SXM, bfloat16, NVLink) as (wall seconds $\times$ GPU count)/3600. PCU-Select pays 144.0 GPU-hours once: 48.0 for feature extraction, 4.8 for gradient-proxy labels, 84.0 for short-update labels, and 7.2 for scorer training. Dividing the 1.51 GPU-hour four-task measurement by four gives an exact average per-task cost of 0.3775 GPU-hours, displayed as 0.38. LESS costs 62.0 per PEFT datastore plus 0.30 per task ranking, hence 63.2 for four tasks. Onboarding a new four-task PEFT therefore costs 1.51 GPU-hours for PCU-Select versus 63.2 for LESS, a $42 \times$ marginal-cost reduction. The shared 12.8 GPU-hour downstream cost applies to each PEFT–task–seed run, so the selection comparison excludes it. For four tasks, break-even is $P^* = 144.0 / [62.0 + 4(0.30 - 0.3775)] = 2.33$ configurations. The pipeline fixes selector-scorer training, task-sketch sampling, and pool ordering at seed 0. Reported SDs vary target-training seeds (0, 1, 2) with the selector and scorer pipeline fixed. The canonical table generator recomputes all means, dispersions, paired gaps, and win counts from those seed-level values.

Two supplementary tables bound the cost claim. Table S6 varies the offline short-update label budget. The 10K default is the smallest evaluated budget that preserves the aggregate quality tie. Smaller budgets reduce offline cost and the break-even point but also reduce quality, making the offline investment a tunable trade-off. Table S7 separates the on-line timing components. At 300K, the instrumented scorer and clustering timers sum to 0.360 GPU-hours. The end-to-end measurement is 0.3775, leaving 0.0175 GPU-hours of uninstrumented overhead, which we retain explicitly rather than assigning to either component. Under a linear datastore-cost extrapolation anchored at LESS’s measured 300K cost, PCU-Select’s end-to-end online cost is approximately $170 \times$ smaller across the reported pool sizes.

Algorithms

Algorithm S1 builds the reusable supervision once. Algorithm S2 applies the scorer to a new target. For seen and L0 targets, this per-target procedure is gradient-free and uses only cached features. L1 and L2 targets additionally require the reduced short-update labels shown in Algorithm S2; after those labels are constructed, fitting and applying the residual head requires no gradient datastore. The offline algorithm amortizes the expensive gradient and label construction.

New registry entry	PCU-Select	LESS
One new task, existing PEFT	0.38	0.30
One new PEFT, four tasks	1.51	63.2

Table S5: The amortization advantage is cross-PEFT, not cross-task. Adding a task to an existing PEFT costs each selector one online-selection pass; adding a PEFT for four tasks forces LESS to rebuild its datastore while PCU-Select reuses the offline scorer. Selection-stage GPU-hours; PCU-Select’s per-task entry rounds the exact average $1.51/4 = 0.3775$.

Labels	Offline GPU-h	Avg. quality	Gap to LESS	Break-even
2.5K	81.0	34.71	−0.43	1.31
5K	102.0	35.02	−0.12	1.65
10K	144.0	35.18	+0.04	2.33

Table S6: The offline short-update label budget sets the quality–break-even trade-off. Short-update labeling is 58% of offline cost, so a smaller budget lowers both offline GPU-hours and the break-even PEFT count, but drops below LESS-level quality. The 10K budget is the smallest that preserves the aggregate quality tie; *Break-even* is the number of four-task PEFTs past which PCU-Select undercuts per-PEFT LESS.

PEFT Capacity Encoding

Table S8 defines the per-family capacity used inside the site weight from the main paper. We map each family’s size knob to a trainable-parameter count at the touched sites, normalize it by d_{model} , and pass it through $\tanh(\eta \log(1 + \text{cap}/d_{\text{model}}))$ so that capacity is monotone but saturating.

Supplementary Analyses

Component ablations. Table S9 reports the full component breakdown summarized in the main analysis. Removing the structured PEFT code produces the largest loss; task conditioning, both label fidelities, uncertainty, and adaptive coverage also contribute.

Cross-PEFT transfer. Table S10 supports the introduction’s 2.0-point cross-family mismatch penalty. All off-diagonal entries fall below their matched diagonals, and cross-family pairs have the largest average gap. Figure S1 shows a complementary selection-side pattern: PCU-Select’s subset overlap falls as configurations diverge, while PEFT-agnostic RDS+ remains exactly 1.000.

Unseen-configuration transfer. Table S11 reports the values plotted in Figure 3 of the main paper. Near-support targets transfer zero-shot. Calibration closes most of the gap for far same-family targets and LN Tuning. For Prefix/P-Tuning, the point estimate falls from 6.97 to 1.99 but leaves a larger residual gap than for L1 or LN Tuning. The 0.18 change from Cal-200 to Cal-500 is descriptive rather than evidence of a reliable difference between the two label budgets.

	Feature Pool ext. (cached)	Scorer forward	Clustering	Residual	PCU online (per p, t)
50K	9.2	0.051	0.010	0.000	0.061
100K	17.6	0.093	0.028	0.000	0.121
300K	48.0	0.249	0.111	0.018	0.378
1M	152.0	0.790	0.420	0.000	1.210

Table S7: Online selection cost versus candidate-pool size on $8 \times \text{H20-96GB}$. Feature extraction is cached. Scorer and clustering columns report existing phase timers. Residual is total minus these timers; at 300K, the revised four-task measurement gives 0.3775 per pair, while phase timers account for 0.360, leaving 0.0175 pending attribution. Online columns show three decimals.

Algorithm S1 Offline supervision (run once)

- 1: **Input:** pool $\mathcal{D}$, seen PEFTs $\mathcal{P}_{\text{seen}}$, tasks with sketches $\{V_t\}$
- 2: Extract sample signatures z_x and site gradients g_x^ω ; cache them $\{O(|\mathcal{D}|) \text{ forward+backward}\}$
- 3: **for all** $(p, t) \in \mathcal{P}_{\text{seen}} \times \text{tasks}$ **do**
- 4: Compute gradient-proxy $u^{\text{lo}}(x, p, t)$ using the main-paper definition
- 5: **end for**
- 6: Sample 10K triples (coverage/uncertainty/boundary); label u^{hi} by short updates
- 7: Train s_ϕ : Phase A proxy pretrain, Phase B joint; keep best held-out-NDCG checkpoint
- 8: **Output:** scorer s_ϕ , cached features

Baseline Specifications

The three named baselines span two relevant axes: whether selection uses gradients and whether its scoring signal depends on the target PEFT. They are not single-factor ablations; the component ablations above isolate individual PCU-Select design choices. We specify each implementation below so that the comparison is auditable.

RDS+ (representation similarity). RDS+ operates on the 768-dimensional joint semantic embedding e_x^{joint} produced by the frozen sentence encoder. This is the semantic component of PCU-Select’s sample representation z_x . RDS+ neither trains the encoder nor uses features learned by the PCU-Select scorer.

RDS+ first standardizes every pool embedding per feature. It forms the query by averaging the 32 task-sketch embeddings and applies the same pool statistics. It then L_2 -normalizes the pool and query vectors and ranks candidates by cosine similarity.

RDS+ is *task-conditioned* through the sketch query but *PEFT-agnostic*: its ranking does not depend on the target configuration. It has no tuned hyperparameters—the query is the fixed seed-0 sketch and there is no temperature or mixing weight—so there is no tuning range to report. The “+” denotes the per-feature standardization applied before cosine, relative to a plain embedding-nearest-neighbor selector.

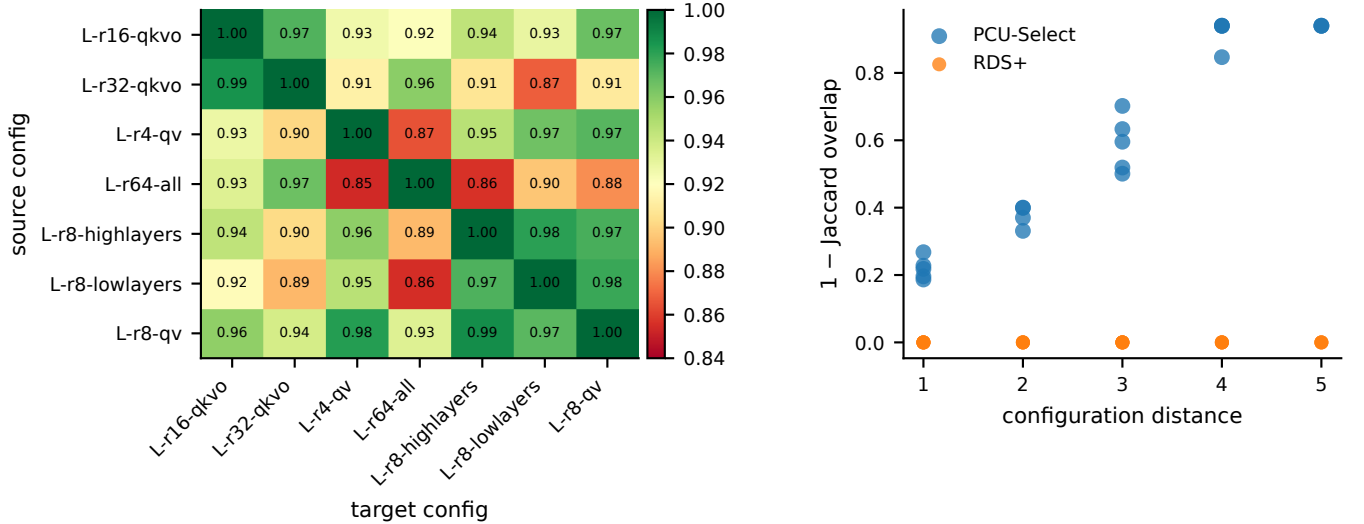


Figure S1: PCU-Select responds to configuration changes. *Left*: Rows denote the PEFT used for selection and columns the PEFT used for target training; each column is normalized by its matched diagonal. The mean normalized off-diagonal drop is 0.068. On the unnormalized scale, the matched and mismatched means are 21.25 and 19.79, respectively, a 1.46-point gap. *Right*: Selection difference, measured as $1 - \text{Jaccard}$, versus the heuristic distance $|\log_2 r_i - \log_2 r_j| + \mathbb{I}[\text{placement differs}] + \mathbb{I}[\text{module set differs}]$. Across the 21 unordered pairs of seven LoRA configurations, the descriptive Spearman correlation is $\rho = 0.969$; leave-one-configuration-out values span $[0.947, 0.972]$. RDS+ lies at zero because, for a fixed task, its ranking is PEFT-independent.

Algorithm S2 Online selection (per target)

- 1: **Input**: target (p^*, t^*) , budget B , cached features, scorer s_ϕ
- 2: Encode $z_{p^*}, z_{t^*}; (\hat{\mu}, \hat{\sigma}) \leftarrow s_\phi(z_x, z_{p^*}, z_{t^*})$ for all x
- 3: **if** reduced calibration labels exist **then**
- 4: Fit and apply the residual head to 200 or 500 labels;
 $\hat{\mu} \leftarrow \hat{\mu}_{\text{cal}}$
- 5: **end if**
- 6: $q(x) \leftarrow \hat{\mu}(x) - \lambda \hat{\sigma}(x)$ for all x
- 7: Cluster the pool; allocate quotas b_k by the main-paper definition
- 8: **Output**: subset $\mathcal{S} = \bigcup_k \{\text{top-}b_k \text{ by } q \text{ in } C_k\}$

Influence (shared-datastore gradient similarity). The Influence baseline builds one shared projected-gradient datastore from the base model’s intervention-site activations. It uses the JL projection and L_2 normalization specified in the reproducibility notes. Each candidate carries signatures g_x^ω at the 24 sites. The task query g_t^ω pools and renormalizes the sketch gradient at each site. Influence scores each candidate with the plain cosine $\sum_\omega \cos(g_x^\omega, g_t^\omega)$ and applies global top- k without site weighting. Its PEFT-agnostic gradient source makes the ranking independent of the target configuration. Influence therefore provides a PEFT-agnostic shared-gradient reference; its comparison with PCU-Select does not by itself isolate the effect of PEFT conditioning.

LESS (per-PEFT gradient influence). LESS is a per-configuration reimplement of gradient-influence selection following LESS (Xia et al. 2024). It applies four steps to each target PEFT. First, it warms up the target adapter

Family	Size knob	cap(p, ω) at a touched site
LoRA	rank r	$r(d_{\text{in}} + d_{\text{out}})$
Adapter	bottleneck b	$2 d_{\text{model}} b$
(IA) ³	(per-vector)	#scaled activations
Prefix/P-Tuning	length L	$L d_{\text{model}}$
LN Tuning	(per-scale)	#RMSNorm scale parameters

Table S8: Per-family capacity mapping. Capacity is the trainable-parameter count contributed at a touched site; z_p encodes operator type separately.

Variant	Metric	Drop	NDCG
Full PCU-Select	19.72±0.22	0.00	0.702
No PEFT code	18.35±0.41	1.37	0.526
Family one-hot only	19.04±0.29	0.68	0.602
No task sketch	18.90±0.31	0.82	0.597
No activation signature	19.24±0.25	0.48	0.629
Gradient-proxy labels only	18.87±0.34	0.85	0.563
Short-update labels only	19.36±0.27	0.36	0.639
No uncertainty penalty	19.44±0.24	0.28	0.690
Global top- k	18.64±0.38	1.08	0.684
Uniform clusters	19.21±0.28	0.51	0.709

Table S9: Ablations on GSM8K and HumanEval with two representative PEFTs at a 10% budget (task-native metric averaged over the two tasks, mean±SD over three seeds). Drop is relative to the full adaptive selector; we measure NDCG against held-out short-update labels.

Source ↓ / Target →	L-r8-qv	L-r16-qkvo	IA3-attnmlp	AD-b64
L-r8-qv	1.00	0.96	0.87	0.93
L-r16-qkvo	0.94	1.00	0.91	0.88
IA3-attnmlp	0.89	0.86	1.00	0.95
AD-b64	0.90	0.92	0.94	1.00

Table S10: Cross-PEFT transfer matrix (motivation study). Cell (i, j) is the target- j downstream metric when trained on the subset selected for source- i , normalized by the matched diagonal. Off-diagonal cells fall below 1.00, and the two directions of a mismatched pair differ by 0.01–0.05 rather than a constant offset. Averaged over all mismatched source-target pairs the gap is 1.87 absolute points (mean matched 21.4 vs. mismatched 19.53 on the GSM8K-scale probe). Cross-family mismatches average a 2.0-point gap, the penalty cited in the introduction. Columns are target PEFTs, rows are the source PEFT used for selection.

Tier	Targets	Ref.	Zero-shot	Cal-200	Cal-500
L0	L-r32-qkvo, AD-b16	LESS	0.34±0.41	–	–
L1	L-r64-all, AD-b256, L-r8-highlayers	LESS	2.08±0.44	0.91±0.29	0.30±0.36
L2-LN	LN	LESS	4.08±0.70	0.65±0.26	0.23±0.42
L2-Prompt	PRE-I16, PT-I32	LESS	6.97±0.34	2.17±0.33	1.99±0.38

Table S11: Performance drop from LESS across GSM8K, HumanEval, and MMLU (LESS minus PCU-Select). Results with repeated-run estimates are paired means over three target-training seeds; each $\pm$ value is the sample SD. Lower is better; zero denotes parity. L0 uses zero-shot only.

for 200 steps to obtain the optimization state. Second, it extracts Adam-preconditioned gradients over the adapter’s own trainable parameters at each checkpoint. Third, it projects the gradients into a target-specific low-dimensional datastore. Finally, it ranks candidates by their projected-gradient influence on the 32-example task sketch and averages across warmup checkpoints.

The first three steps build the datastore over the target adapter’s own trainable parameters. The datastore is therefore PEFT-specific but task-independent. LESS constructs it once per PEFT and reuses it for every task sketch. Only the final projected-gradient scoring step is task-specific.

It costs about 0.30 GPU-hours per task, so 1.20 covers all four task sketches for one configuration. The per-target cost of a PEFT configuration is therefore 62.0 (datastore) + 1.20 (four per-task rankings) = 63.2 GPU-hours, already spanning the full four-task evaluation; the five-PEFT registry cost is $5 \times 63.2 = 316.0$. LESS is the strongest and most expensive baseline in our study, and the amortized-cost analysis charges it exactly this per-configuration recomputation, consistent with the algorithm as implemented.

Baseline configuration and tuning. Random has no hyperparameters. RDS+ uses the fixed representation-similarity rule above with the shared 32-example sketch as its only task input. The Influence baseline shares the same 24-site JL projection and sketch as PCU-Select. We fix every baseline’s hyperparameters before test-metric evaluation.

References

Austin, J.; Odena, A.; Nye, M.; Bosma, M.; Michalewski, H.; Dohan, D.; Jiang, E.; Cai, C.; Terry, M.; Le, Q. V.; and Sutton, C. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732*.

Chaudhary, S. 2023. Code Alpaca: An Instruction-Following LLaMA Model for Code Generation. GitHub repository.

Chen, M.; et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.

Clark, J. H.; Choi, E.; Collins, M.; Garrette, D.; Kwiatkowski, T.; Nikolaev, V.; and Palomaki, J. 2020. TyDi QA: A Benchmark for Information-Seeking Question Answering in Typologically Diverse Languages. *Transactions of the Association for Computational Linguistics*, 8: 454–470.

Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.

Conover, M.; Hayes, M.; Mathur, A.; Xie, J.; Wan, J.; Shah, S.; Ghodsi, A.; Wendell, P.; Zaharia, M.; and Xin, R. 2023. Free Dolly: Introducing the World’s First Truly Open Instruction-Tuned LLM. Databricks Blog.

Dao, T. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations*.

Hendrycks, D.; Burns, C.; Basart, S.; Zou, A.; Mazeika, M.; Song, D.; and Steinhardt, J. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*.

Iverson, H.; Wang, Y.; Pyatkin, V.; Lambert, N.; Peters, M. E.; Dasigi, P.; Jang, J.; Wadden, D.; Smith, N. A.; Beltagy, I.; and Hajishirzi, H. 2023. Camels in a Changing Climate: Enhancing LM Adaptation with Tulu 2. *arXiv preprint arXiv:2311.10702*.

Johnson, W. B.; and Lindenstrauss, J. 1984. Extensions of Lipschitz Mappings into a Hilbert Space. In *Conference in Modern Analysis and Probability*, volume 26 of *Contemporary Mathematics*, 189–206. American Mathematical

Society.

Köpf, A.; Kilcher, Y.; von Rütte, D.; Anagnostidis, S.; Tam, Z. R.; Stevens, K.; Barhoum, A.; Nguyen, D.; Stanley, O.; Nagyfi, R.; ES, S.; Suri, S.; Glushkov, D.; Dantuluri, A.; Maguire, A.; Schuhmann, C.; Nguyen, H.; and Mattick, A. 2023. OpenAssistant Conversations—Democratizing Large Language Model Alignment. In *Advances in Neural Information Processing Systems*, volume 36.

Lian, W.; Goodson, B.; Pentland, E.; Cook, A.; Vong, C.; and Teknium. 2023. OpenOrca. Hugging Face dataset repository.

Longpre, S.; Hou, L.; Vu, T.; Webson, A.; Chung, H. W.; Tay, Y.; Zhou, D.; Le, Q. V.; Zoph, B.; Wei, J.; and Roberts, A. 2023. The Flan Collection: Designing Data and Methods for Effective Instruction Tuning. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, 22631–22648. PMLR.

Perez, E.; Strub, F.; de Vries, H.; Dumoulin, V.; and Courville, A. 2018. FiLM: Visual Reasoning with a General Conditioning Layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 3942–3951.

Sculley, D. 2010. Web-Scale K-Means Clustering. In *Proceedings of the 19th International Conference on World Wide Web*, 1177–1178. ACM.

Touvron, H.; et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*.

Wang, Q.; Ruan, Y.-P.; Zuo, Y.; and Li, T. 2022. Parameter-Efficient Tuning on Layer Normalization for Pre-trained Language Models. *arXiv preprint arXiv:2211.08682*.

Xia, M.; Malladi, S.; Gururangan, S.; Arora, S.; and Chen, D. 2024. LESS: Selecting Influential Data for Targeted Instruction Tuning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 54104–54132. PMLR.

Xu, C.; Sun, Q.; Zheng, K.; Geng, X.; Zhao, P.; Feng, J.; Tao, C.; Lin, Q.; and Jiang, D. 2024. WizardLM: Empowering Large Pre-Trained Language Models to Follow Complex Instructions. In *International Conference on Learning Representations*.